DJANGO MSE 2 Imp Questions (U3, U4)

Unit 3

1.Create a Django view to handle a Student Registration form and render the submitted data to a template.

a) Implement manual rendering by passing individual fields to the template.
b) Implement automatic rendering by passing the form instance to the template.
c) Write complete code for views.py and the template (.html) for both methods.

(a) Manual Rendering (Passing Individual Fields)

✅ views.py

```
from django.shortcuts import render
from django import forms

# Create Form
class StudentForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    age = forms.IntegerField()

def student_register(request):
    if request.method == "POST":
        name = request.POST.get('name')
        email = request.POST.get('email')
        age = request.POST.get('age')

        return render(request, 'result.html', {
            'name': name,
            'email': email,
            'age': age
        })
    return render(request, 'register.html')
```

✅ register.html (Form Page)

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
</head>
<body>
  <h2>Register Student</h2>
  <form method="post">
    {% csrf_token %}
    Name: <input type="text" name="name"><br><br>
    Email: <input type="email" name="email"><br><br>
    Age: <input type="number" name="age"><br><br>
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

✅ result.html (Display Data)

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Data</title>
</head>
<body>
  <h2>Submitted Data</h2>
  <p>Name: {{ name }}</p>
  <p>Email: {{ email }}</p>
  <p>Age: {{ age }}</p>
</body>
</html>
```

(b) Automatic Rendering (Passing Form Instance)

✅ views.py

```
from django.shortcuts import render
from django import forms
```

```
# Create Form
class StudentForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    age = forms.IntegerField()

def student_register(request):
    form = StudentForm()

    if request.method == "POST":
        form = StudentForm(request.POST)
        if form.is_valid():
            return render(request, 'result.html', {
                'form': form
            })

    return render(request, 'register.html', {'form': form})
```

✅ register.html (Auto Form Rendering)

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>
    <h2>Register Student</h2>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

✅ result.html (Display Form Data)

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Data</title>
```

```
</head>
<body>
  <h2>Submitted Data</h2>
  <p>Name: {{ form.cleaned_data.name }}</p>
  <p>Email: {{ form.cleaned_data.email }}</p>
  <p>Age: {{ form.cleaned_data.age }}</p>
</body>
</html>
```

🔑 Key Difference (Important for Exam)

Manual Rendering	Automatic Rendering
Data passed individually	Entire form passed
More control	Less code
Uses request.POST.get()	Uses form.cleaned_data
HTML written manually	Django renders form automatically

✍️ Short Conclusion (Write in exam ending)

Manual rendering gives full control over form fields, while automatic rendering simplifies form handling by using Django's form system. Both approaches help process and display submitted data efficiently.

2. Create a Django form for a Student Registration with at least four fields using different form widgets (TextInput, EmailInput, DateInput, select). Customize each widget using attr and write the complete forms.py code.

1) direct questions :like about static files.
2) one of sample question
3) direct questions like template tags and filters
4) one of sample question

✅ forms.py

```
from django import forms
```

```
class StudentForm(forms.Form):

    name = forms.CharField(
        max_length=100,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your name'
        })
    )

    email = forms.EmailField(
        widget=forms.EmailInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your email'
        })
    )

    dob = forms.DateField(
        widget=forms.DateInput(attrs={
            'class': 'form-control',
            'type': 'date'
        })
    )

    course = forms.ChoiceField(
        choices=[
            ('bca', 'BCA'),
            ('mca', 'MCA'),
            ('btech', 'B.Tech')
        ],
        widget=forms.Select(attrs={
            'class': 'form-control'
        })
    )
```

✅ Explanation (for exam writing)

- **TextInput** → Used for name field
- **EmailInput** → Used for email validation
- **DateInput** → Used to select date (with type="date")
- **Select** → Used for dropdown choices

- attrs is used to:
    - Add CSS classes (form-control)
    - Add placeholders
    - Customize appearance of form fields

✍️ Short 5-Mark Answer

A Django form can be created using different widgets like TextInput, EmailInput, DateInput, and Select. Each field can be customized using the attrs attribute to add CSS classes, placeholders, and HTML properties. This improves the appearance and usability of the form.

1) Direct Question: Static Files in Django

What are Static Files?

Static files are files like **CSS, JavaScript, and Images** used to design and enhance the frontend of a web application.

Steps to Use Static Files

1. Add in settings.py:

```
STATIC_URL = '/static/'
```

2. Create folder structure:

```
project/
  static/
    css/
    js/
    images/
```

3. Load static in template:

```
{% load static %}
<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Key Points

- Used for styling and UI
- Django serves them in development
- In production, handled separately

✍️ Short Answer (Exam)

Static files in Django include CSS, JavaScript, and images. They are managed using the static folder and accessed in templates using {% load static %} and {% static %} tags.

---

2) Sample Question

Q: Create a Django page to display student details using templates

views.py

```
def student_view(request):
    data = {
        'name': 'Akash',
        'age': 22,
        'course': 'MCA'
    }
    return render(request, 'student.html', data)
```

---

student.html

```
<!DOCTYPE html>
<html>
<body>
    <h2>Student Details</h2>
    <p>Name: {{ name }}</p>
    <p>Age: {{ age }}</p>
    <p>Course: {{ course }}</p>
</body>
</html>
```

---

3) Direct Question: Template Tags and Filters

Template Tags

Used to perform logic in templates.

Examples:

```
{% if user %}
  Hello User
{% endif %}

{% for i in list %}
  {{ i }}
{% endfor %}
```

Template Filters

Used to modify variables.

Examples:

```
{{ name|upper }}
{{ name|lower }}
{{ text|truncatewords:2 }}
```

Common Filters

- upper → Convert to uppercase
- lower → Convert to lowercase
- length → Count items
- truncatewords → Limit words

✍️ Short Answer

Template tags are used for logic like loops and conditions, while filters modify data output such as converting text to uppercase or limiting words.

---

4) Sample Question

Q: Display a list of students using loop and apply filters

views.py

```
def student_list(request):
    students = ['akash', 'rahul', 'sneha']
    return render(request, 'list.html', {'students': students})
```

---

list.html

```
<!DOCTYPE html>
<html>
<body>
    <h2>Student List</h2>

    {% for student in students %}
        <p>{{ student|upper }}</p>
    {% endfor %}

    <p>Total Students: {{ students|length }}</p>
</body>
</html>
```

🔥 Final Tip (Important for Exam)

- Static files → **frontend resources**
- Template tags → **logic**
- Filters → **data modification**
- Always write **example + code** → gets full marks

Unit 4

1. How to activate the admin page

Steps to Activate Django Admin Page

1. **Create a Django project**
   Use:

   django-admin startproject project_name

2. **Create and add an app to the project**

   python manage.py startapp app_name

   Then add the app inside settings.py under INSTALLED_APPS.

3. **Enable admin in settings**
   Make sure 'django.contrib.admin' is included in INSTALLED_APPS.
4. **Run migrations**

   python manage.py makemigrations
   python manage.py migrate

5. **Create a superuser (admin login)**

   python manage.py createsuperuser

   Enter username, email, and password.

6. **Start the development server**

   python manage.py runserver

7. **Open admin page in browser**
   Go to:

   http://127.0.0.1:8000/admin/

   Login using the superuser credentials.

Key Point (for exams)

- Django admin is a **built-in interface** used for managing database data.
- It supports **CRUD operations (Create, Read, Update, Delete)**.
- It is accessible only to **authorized admin users**.

2. Class-Based Views (CBV) in Django

Class-Based Views (CBV) in Django

Definition:
Class-Based Views (CBV) are views written using Python classes instead of functions. They handle different HTTP requests using methods like get(), post(), etc.

How CBV Works

1. User sends a request
2. URL is mapped to the view
3. as_view() method converts class into a callable function
4. Appropriate method (get(), post()) is executed
5. Response is returned to the user

Example of CBV

```
from django.views import View
from django.http import HttpResponse

class MyView(View):
    def get(self, request):
        return HttpResponse("Hello CBV")

    def post(self, request):
        return HttpResponse("POST request")
```

Key Points

- Uses **classes instead of functions**
- Supports multiple HTTP methods in one class
- Improves **code reusability and organization**
- Uses as_view() to convert class into a view

Short 5-Mark Version (for exams)

Class-Based Views (CBV) in Django are views defined using Python classes. They handle HTTP requests through methods like get() and post(). When a request is made, Django calls the as_view() method, which converts the class into a callable view function. CBVs improve code reusability and make applications more structured compared to function-based views.